

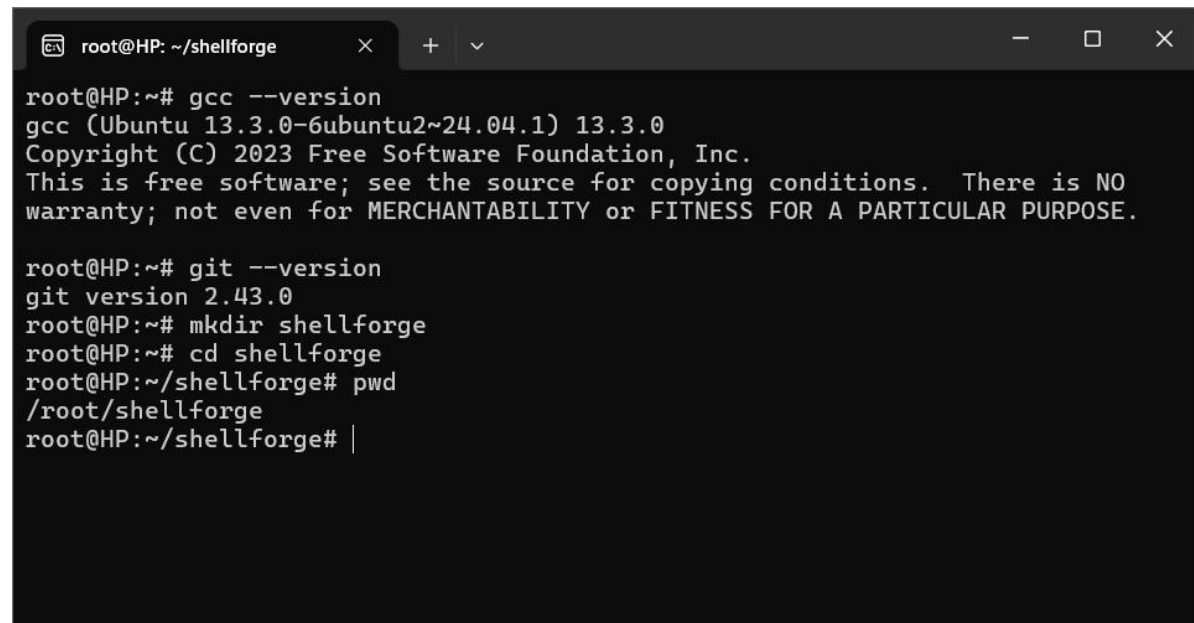
Name:P Neeharika

Roll No: 2520090233

Session 1

Task1 – Linux Environment Setup

Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

A terminal window titled 'root@HP: ~/shellforge' with standard window controls. The terminal shows the following commands and output:

```
root@HP:~# gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@HP:~# git --version
git version 2.43.0
root@HP:~# mkdir shellforge
root@HP:~# cd shellforge
root@HP:~/shellforge# pwd
/root/shellforge
root@HP:~/shellforge# |
```

```

/root/shellforge
root@HP:~/shellforge# git init
hint: Using 'master' as the name for the initial branch. This default branch
name
hint: is subject to change. To configure the initial branch name to use in al
l
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /root/shellforge/.git/
root@HP:~/shellforge# git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
root@HP:~/shellforge#

```

Task 2:

Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing

```

root@HP:~# ls
demo1.c  fork_example  neeha    process.c  sri        xls.txt
demo2    fork_example.c neetu    session1.c states
demo2.c  nano.867.save process  shellforge states.c
root@HP:~# gcc session1.c -o session1
root@HP:~# ./session1
This is the parent process.
This is the child process.
Parent PID: 596
Child PID: 597
root@HP:~# |

```

```

Parent PID: 596
Child PID: 597
root@HP:~# ps -f
UID      PID     PPID  C  STIME TTY          TIME CMD
root      568     563  0  09:57 pts/0        00:00:00 -bash
root      601     568  50 10:01 pts/0        00:00:00 ps -f
root@HP:~# |

root@HP:~# ps -f
UID      PID     PPID  C  STIME TTY          TIME CMD
root      568     563  0  09:57 pts/0        00:00:00 -bash
root      601     568  50 10:01 pts/0        00:00:00 ps -f
root@HP:~# pstree -p
systemd(1)─agetty(218)
            └─cron(180)
            └─dbus-daemon(181)
            └─init-systemd(Ub(2))─SessionLeader(562)─Relay(568)(563)─ba+
                                └─init(7)─{init}(8)
                                └─login(329)─bash(406)
                                └─{init-systemd(Ub)}(9)
            └─rsyslogd(220)─{rsyslogd}(235)
                            └─{rsyslogd}(236)
                            └─{rsyslogd}(237)
            └─systemd(377)─(sd-pam)(380)
            └─systemd-journal(62)
            └─systemd-logind(195)
            └─systemd-resolve(121)
            └─systemd-timesyn(122)─{systemd-timesyn}(169)
            └─systemd-udev(114)─(udev-worker)(602)
                                └─(udev-worker)(603)
            └─unattended-upgr(272)─{unattended-upgr}(278)
            └─wsl-pro-service(208)─{wsl-pro-service}(260)
                                    └─{wsl-pro-service}(261)
                                    └─{wsl-pro-service}(262)
                                    └─{wsl-pro-service}(263)
                                    └─{wsl-pro-service}(264)
                                    └─{wsl-pro-service}(265)
                                    └─{wsl-pro-service}(274)

root@HP:~#

```

Session 1 Observation:

The session provided practical experience in setting up the Linux development environment and understanding fundamental operating-system concepts related to processes. The first task established the required development tools and project structure, while the second task provided hands-on experience with process creation, program execution, parent-child relationships, process trees, and system-call tracing. Overall, the activities helped connect the theoretical concepts of operating systems with their practical implementation.